# ShadowStack

Predictive Cloud Cost Optimizer

# Dataset & Database Schema Design Document

| | |
|---|---|
| **Document Version:** | 1.0 |
| **Status:** | Final Draft |
| **Date:** | February 24, 2026 |
| **Course:** | BCA 6th Semester — 3 Credits Project |
| **Database:** | PostgreSQL 15 |
| **Reviewed by:** | Aniruddha Bhide (Backend Architect) |


| Name | Student ID | Role |
|---|---|---|
| Aniruddha Bhide | CU23BSC001A | Project Lead & Backend Architect |
| Sanay Krishna | CU23BCA0049A | Frontend Developer & Visualization Engineer |
| Arpita M | CU23BCA0011A | Documentation & Quality Support Engineer |


*Sprint 1 deliverable (PBI-012). Primary reference for all data engineering work in Sprint 2. Defines all database tables, column specifications, relationships, indexes, and the synthetic dataset generation strategy for ShadowStack.*

# Contents

**Revision History** **2**

**1 Introduction** **2**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Definitions . . . 2

**2 Database Overview** **2**
- 2.1 DBMS Configuration . . . 3
- 2.2 Table Summary and Relationships . . . 3
- 2.3 Entity Relationship Diagram . . . 3

**3 Table Specifications** **4**
- 3.1 `model_metrics` . . . 4
- 3.2 `usage_data` . . . 4
- 3.3 `predictions` . . . 5

**4 Synthetic Dataset Specification** **7**
- 4.1 Overview . . . 7
- 4.2 Column Definitions and Generation Rules . . . 7
- 4.3 Mocked Unit Prices (`config/pricing.yaml`) . . . 7
- 4.4 Dataset Size, Split and Reproducibility . . . 8
- 4.5 Data Quality Checks . . . 8

**5 ML Feature Engineering Specification** **9**
- 5.1 Feature Set . . . 9
- 5.2 Preprocessing Pipeline Steps . . . 9

**6 Constraints, Assumptions and Limitations** **9**

**A Complete DDL — All Tables** **11**

**B Sample Data — `model_metrics`** **12**

## Revision History

| Version | Date | Author | Description |
|---|---|---|---|
| 0.1 | Feb 10, 2026 | Arpita M | Initial draft — table list and column names |
| 0.2 | Feb 17, 2026 | Aniruddha Bhide | Data types, constraints, FK relationships |
| 0.3 | Feb 21, 2026 | Arpita M | Synthetic dataset strategy and index section |
| 1.0 | Feb 24, 2026 | Arpita M | Final review and SQL DDL; approved by Aniruddha |

# 1 Introduction

## 1.1 Purpose

This document specifies the complete database schema and dataset design for ShadowStack — Predictive Cloud Cost Optimizer. It defines every table, column, data type, constraint, index, and relationship used by the system, and describes the strategy for generating the synthetic ML training dataset.

## 1.2 Scope

The schema covers the three primary application tables: `model_metrics` (ML model versions and evaluation results), `usage_data` (historical and synthetic usage records, primary ML training source), and `predictions` (one record per PR analysis run). A supplementary synthetic dataset specification is included for the CSV file generated in Sprint 2.

## 1.3 Definitions

| Term | Definition |
|---|---|
| **PK** | Primary Key — uniquely identifies each row |
| **FK** | Foreign Key — references a primary key in another table |
| BIGSERIAL | PostgreSQL auto-incrementing 64-bit integer sequence |
| TIMESTAMPTZ | Timestamp with time zone, stored in UTC |
| NUMERIC(p,s) | Exact fixed-point number with precision $p$ and scale $s$ |
| MAE / RMSE / $R^2$ | Mean Absolute Error / Root Mean Square Error / Coefficient of Determination |
| DDL | Data Definition Language — SQL statements defining the schema |

# 2 Database Overview

## 2.1 DBMS Configuration

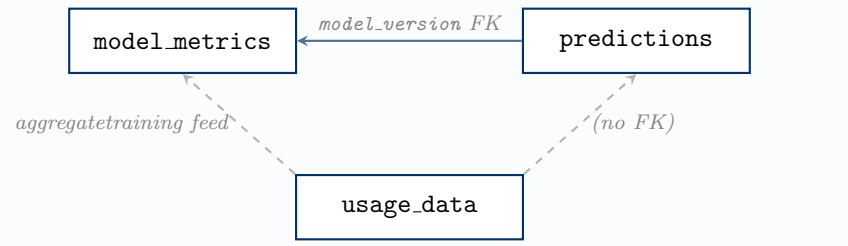
| Property | Value |
|---|---|
| DBMS | PostgreSQL 15 |
| Database Name | `shadowstack_db` |
| Port / Schema | 5432 / `public` |
| Character Encoding | UTF-8 |
| Migration Tool | Alembic 1.13+ via SQLAlchemy 2.x |
| Deployment | Docker container (`postgres:15-alpine`), volume `shadowstack_pgdata` |
| Connection (internal) | `postgresql://user:pass@db:5432/shadowstack_db` |

## 2.2 Table Summary and Relationships

| Table | Sprint | Purpose |
|---|---|---|
| `model_metrics` | S1 | ML model versions and training evaluation results |
| `usage_data` | S1 | Historical and synthetic cloud usage; primary ML training source |
| `predictions` | S1 | Every cost prediction output from the analysis pipeline |

**Relationships:** `predictions.model_version` → `model_metrics.model_version` (FK, CASCADE on update, RESTRICT on delete). `usage_data` feeds the ML training pipeline in aggregate — no row-level FK to `predictions`.

## 2.3 Entity Relationship Diagram




*Solid arrow = FK constraint. Dashed = logical data dependency only.*

## 3 Table Specifications

### 3.1 model_metrics

Stores training evaluation results and versioning metadata for every ML model. Created first because `predictions` holds a FK reference to it.

| Column | Type | Key | Null | Default | Description |
|---|---|---|---|---|---|
| `id` | BIGSERIAL | **PK** | NO | auto | Auto-increment PK |
| `model_version` | VARCHAR(50) | UQ | NO | — | Unique version string, e.g. `v2_random_forest` |
| `model_type` | VARCHAR(50) | — | NO | — | `linear_regression`, `arima`, or `random_forest` |
| `mae` | NUMERIC(10,6) | — | NO | — | Mean Absolute Error on held-out test set ($\geq 0$) |
| `rmse` | NUMERIC(10,6) | — | NO | — | Root Mean Square Error ($\geq 0$) |
| `r2_score` | NUMERIC(10,6) | — | NO | — | $R^2$ coefficient of determination |
| `mae_pct` | NUMERIC(7,4) | — | YES | NULL | MAE as % of mean actual cost; used for the <15% target check |
| `train_samples` | INTEGER | — | NO | — | Rows used for training ($> 0$) |
| `test_samples` | INTEGER | — | NO | — | Rows held out for evaluation ($> 0$) |
| `feature_count` | INTEGER | — | YES | NULL | Number of input features |
| `hyperparameters` | TEXT | — | YES | NULL | JSON string of key hyperparameter values |
| `model_file_path` | VARCHAR(300) | — | NO | — | Path to serialised `.pkl` file |
| `is_best` | BOOLEAN | — | NO | FALSE | TRUE = currently deployed model; only one row TRUE at a time |
| `trained_at` | TIMESTAMPTZ | — | NO | NOW() | Training completion timestamp (UTC) |
| `notes` | TEXT | — | YES | NULL | Free-text notes on the training run |

**Indexes:** `idx_mm_model_version` (UNIQUE), `idx_mm_is_best`, `idx_mm_model_type`.

### 3.2 usage_data

Stores all cloud resource usage records — synthetic (Sprint 2 generator) and real (imported logs). Primary ML training data source; also queried by `GET /api/costs/history`.

| Column | Type | Key | Null | Default | Description |
|---|---|---|---|---|---|
| `id` | BIGSERIAL | **PK** | NO | auto | Auto-increment PK |

| Column | Type | Key | Null | Default | Description |
|---|---|---|---|---|---|
| `timestamp` | TIMESTAMPTZ | — | NO | — | UTC measurement time; generated past date for synthetic records |
| `service_name` | VARCHAR(100) | — | NO | — | `compute`, `storage`, `network`, `database`, or `cache` |
| `cost_usd` | NUMERIC(12,4) | — | NO | — | Actual or synthetic cost in USD ($\geq 0$) |
| `resource_type` | VARCHAR(100) | — | YES | NULL | Sub-type, e.g. `t3.medium`, `S3-standard` |
| `region` | VARCHAR(50) | — | YES | NULL | Cloud region, e.g. `us-east-1` |
| `complexity_score` | NUMERIC(5,2) | — | YES | NULL | Associated code complexity (1.00–10.00); NULL for historical imports |
| `resource_units` | NUMERIC(12,4) | — | YES | NULL | Quantity consumed (GB-hours, vCPU-hours, etc.) |
| `is_synthetic` | BOOLEAN | — | NO | TRUE | TRUE = generated record; FALSE = real imported data |
| `created_at` | TIMESTAMPTZ | — | NO | NOW() | Row insertion timestamp |

**Constraints:** `cost_usd` $\geq$ `0`; `complexity_score` BETWEEN 1.0 AND 10.0 OR NULL; `service_name` IN (`compute`, `storage`, `network`, `database`, `cache`).
**Indexes:** `idx_ud_timestamp`, `idx_ud_service_name`, `idx_ud_ts_service` (composite), `idx_ud_synthetic`.

### 3.3 predictions

One row per PR analysis run (or direct `POST /api/predict` call). Source of truth for all dashboard cost charts and PR comment content.

| Column | Type | Key | Null | Default | Description |
|---|---|---|---|---|---|
| `id` | BIGSERIAL | **PK** | NO | auto | Auto-increment PK |
| `pr_number` | INTEGER | — | YES | NULL | GitHub PR number; NULL for direct API calls |
| `repository` | VARCHAR(200) | — | YES | NULL | Full name `owner/repo` |
| `pr_sha` | VARCHAR(40) | — | YES | NULL | HEAD commit SHA at analysis time |
| `complexity_score` | NUMERIC(5,2) | — | NO | — | Cyclomatic complexity (1.00–10.00) from AST analysis |
| `function_count` | INTEGER | — | YES | NULL | Functions detected |
| `class_count` | INTEGER | — | YES | NULL | Classes detected |

| Column | Type | Key | Null | Default | Description |
|---|---|---|---|---|---|
| import_count | INTEGER | — | YES | NULL | Imports detected |
| loop_count | INTEGER | — | YES | NULL | Loop constructs (for+while) detected |
| branch_count | INTEGER | — | YES | NULL | Conditional branches detected |
| predicted_cost_usd | NUMERIC(12,4) | — | NO | — | Predicted total monthly cost (USD, $\geq 0$) |
| baseline_cost_usd | NUMERIC(12,4) | — | YES | NULL | Pre-change baseline cost; NULL if not established |
| cost_delta_usd | NUMERIC(12,4) | — | YES | NULL | predicted − baseline; positive = cost increase |
| compute_cost_usd | NUMERIC(12,4) | — | YES | NULL | Itemised compute cost (USD/month) |
| storage_cost_usd | NUMERIC(12,4) | — | YES | NULL | Itemised storage cost (USD/month) |
| network_cost_usd | NUMERIC(12,4) | — | YES | NULL | Itemised network cost (USD/month) |
| model_version | VARCHAR(50) | **FK** | NO | — | References model_metrics.model_version |
| model_type | VARCHAR(50) | — | NO | — | linear_regression, arima, or random_forest |
| recommendation | TEXT | — | YES | NULL | Optimisation recommendation text |
| pr_comment_posted | BOOLEAN | — | NO | FALSE | TRUE if GitHub PR comment was posted successfully |
| created_at | TIMESTAMPTZ | — | NO | NOW() | Prediction timestamp (UTC) |

**Constraints:** complexity_score BETWEEN 1.0 AND 10.0; predicted_cost_usd $\geq$ 0; model_type IN allowed set; FK on model_version (CASCADE UPDATE, RESTRICT DELETE).
**Indexes:** idx_pred_created_at, idx_pred_pr_number, idx_pred_repository, idx_pred_model_version.

# 4 Synthetic Dataset Specification

## 4.1 Overview

The ML pipeline requires labelled training data mapping code complexity to monthly infrastructure costs. Since real production data is unavailable, the Sprint 2 generation script (`scripts/generate_dataset.py`) produces at least 500 synthetic records with a fixed seed for full reproducibility. Data is stored in both the `usage_data` table (`is_synthetic = TRUE`) and `data/synthetic_usage_data.csv`.

## 4.2 Column Definitions and Generation Rules

| Field | Type | Range | Generation Rule |
|---|---|---|---|
| `timestamp` | datetime | Past 12 mo | Uniformly sampled UTC timestamps over the past 365 days |
| `service_name` | string | 5 values | Weighted draw: compute 40%, storage 25%, network 20%, database 10%, cache 5% |
| `complexity_score` | float | 1.0–10.0 | Beta(2,5) distribution scaled to [1,10]; realistic left-skewed complexity |
| `resource_units` | float | 0–1000 | `complexity_score` × `service_multiplier` × $\mathcal{N}(1, 0.15)$ |
| `cost_usd` | float | 0.01–5000 | **Label.** `resource_units` × `unit_price[service]` $+ \varepsilon$; $\varepsilon \sim \mathcal{N}(0, 2)$ |
| `resource_type` | string | per service | Sampled from per-service resource type list (e.g., t3.micro, S3-standard) |
| `region` | string | 3 values | Uniform draw: `us-east-1`, `eu-west-1`, `ap-south-1` |
| `is_synthetic` | boolean | TRUE | Always TRUE for generated records |

## 4.3 Mocked Unit Prices (`config/pricing.yaml`)

| Service | Resource Type | Unit Price (USD) | Basis |
|---|---|---|---|
| compute | t3.micro | $0.0104 | per vCPU-hour |
| compute | t3.medium | $0.0416 | per vCPU-hour |
| compute | t3.large | $0.0832 | per vCPU-hour |
| storage | S3-standard | $0.023 | per GB-month |
| storage | EBS-gp3 | $0.08 | per GB-month |
| network | egress | $0.09 | per GB transferred |
| database | RDS-postgres | $0.115 | per instance-hour |
| cache | ElastiCache | $0.034 | per node-hour |

### 4.4 Dataset Size, Split and Reproducibility

| Parameter | Value | Notes |
|---|---|---|
| Minimum rows | 500 | PBI-016 acceptance criterion |
| Target rows | 1,000 | Recommended for robust model training |
| Train / test split | 80 / 20 | Stratified by `service_name` |
| Random seed | 42 | Fixed; identical output on every run |
| CSV output path | `data/synthetic_usage_data.csv` — committed to repository | |

### 4.5 Data Quality Checks

The following checks run immediately after generation, before loading into PostgreSQL:

| # | Check | Validation Rule |
|---|---|---|
| 1 | Row count | Total rows $\geq$ 500 |
| 2 | No nulls | `timestamp`, `service_name`, `cost_usd`, `complexity_score`, `resource_units` must have zero NULLs |
| 3 | Cost range | All `cost_usd` $\geq$ 0 |
| 4 | Complexity range | All `complexity_score` in [1.0, 10.0] |
| 5 | Service coverage | Each of the 5 service categories has $\geq$ 10 records |
| 6 | Timestamp range | All timestamps within the past 365 days |
| 7 | Cost variance | Std dev of `cost_usd` $>$ 5 (non-trivial distribution) |
| 8 | Signal check | Pearson correlation(`complexity_score`, `cost_usd`) $>$ 0.3 |

# 5 ML Feature Engineering Specification

## 5.1 Feature Set

The feature extraction pipeline (PBI-022) converts raw code metrics and usage data into normalised feature vectors. The same pipeline is used for both training and inference (serialised to `models/feature_pipeline.pkl` via `joblib`).

| # | Feature Name | Source | Type | Description |
|---|---|---|---|---|
| 1 | `complexity_score_n` | AST analysis | float [0,1] | MinMax-normalised cyclomatic complexity |
| 2 | `function_count_n` | AST analysis | float [0,1] | Normalised function count |
| 3 | `class_count_n` | AST analysis | float [0,1] | Normalised class count |
| 4 | `import_count_n` | AST analysis | float [0,1] | Normalised import count |
| 5 | `loop_count_n` | AST analysis | float [0,1] | Normalised loop count |
| 6 | `branch_count_n` | AST analysis | float [0,1] | Normalised branch count |
| 7 | `resource_units_n` | usage_data | float [0,1] | Normalised resource units |
| 8 | `svc_compute` | usage_data | int {0,1} | One-hot: service = compute |
| 9 | `svc_storage` | usage_data | int {0,1} | One-hot: service = storage |
| 10 | `svc_network` | usage_data | int {0,1} | One-hot: service = network |
| 11 | `svc_database` | usage_data | int {0,1} | One-hot: service = database |
| 12 | `svc_cache` | usage_data | int {0,1} | One-hot: service = cache |
| 13 | `complexity_x_units` | Engineered | float [0,1] | Interaction term: feature 1 × feature 7 |

**Target variable (label):** `cost_usd` — StandardScaler applied during training; inverse-transformed to return real USD values at inference.

## 5.2 Preprocessing Pipeline Steps

1. **Imputation:** Median imputation on all numeric columns before scaling.
2. **MinMax scaling:** Features 1–7 scaled to [0, 1].
3. **One-hot encoding:** `service_name` → 5 binary columns (features 8–12); all 5 retained for interpretability (no drop-first).
4. **Interaction term:** Feature 13 computed after scaling.
5. **Target scaling:** StandardScaler on `cost_usd`; inverse-transformed at inference.
6. **Serialisation:** Full pipeline saved to `models/feature_pipeline.pkl`; loaded identically at inference time to guarantee consistent transformations.

# 6 Constraints, Assumptions and Limitations

**Constraints:**

- All monetary values stored in **USD**; currency conversion is out of scope.
- Pricing data is **mocked and static** (YAML config); live API integration is post-MVP.

- Only **Python** source files are supported; schema fields reflect Python-specific AST constructs.
- `hyperparameters` stores a raw JSON string (not JSONB) for Alembic migration simplicity.
- Tables designed for up to **10,000 records** under local Docker resource limits.

**Assumptions:**

- All timestamps stored in **UTC**; frontend handles local time zone conversion.
- Dataset generator runs once in Sprint 2 with seed 42; reruns produce identical data.
- Complexity 1.0 = minimal (single-function scripts); 10.0 = maximum analysed complexity.
- `is_best` in `model_metrics` is managed exclusively by `scripts/evaluate_models.py`.
- Alembic runs `upgrade head` automatically at API container startup — no manual migration step required.

**Known Limitations:**

- No multi-tenancy — all predictions share a single namespace.
- No soft deletes — `deleted_at` columns are not included in the MVP schema.
- No application-level audit trail beyond the `created_at` timestamps.

# A Complete DDL — All Tables

Listing 1: ShadowStack Complete Schema DDL (PostgreSQL 15)

```sql
-- ============================================================
-- ShadowStack Database Schema  |  v1.0  |  2026-02-24
-- ============================================================

-- Table 1: model_metrics (must be created before predictions)
CREATE TABLE IF NOT EXISTS model_metrics (
    id                BIGSERIAL       PRIMARY KEY,
    model_version     VARCHAR(50)     NOT NULL UNIQUE,
    model_type        VARCHAR(50)     NOT NULL,
    mae               NUMERIC(10, 6)  NOT NULL,
    rmse              NUMERIC(10, 6)  NOT NULL,
    r2_score          NUMERIC(10, 6)  NOT NULL,
    mae_pct           NUMERIC(7, 4),
    train_samples     INTEGER         NOT NULL,
    test_samples      INTEGER         NOT NULL,
    feature_count     INTEGER,
    hyperparameters   TEXT,
    model_file_path   VARCHAR(300)    NOT NULL,
    is_best           BOOLEAN         NOT NULL DEFAULT FALSE,
    trained_at        TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    notes             TEXT,
    CONSTRAINT chk_mm_model_type CHECK (model_type IN
        ('linear_regression','arima','random_forest')),
    CONSTRAINT chk_mm_mae_nonneg  CHECK (mae  >= 0),
    CONSTRAINT chk_mm_rmse_nonneg CHECK (rmse >= 0),
    CONSTRAINT chk_mm_train_pos   CHECK (train_samples > 0),
    CONSTRAINT chk_mm_test_pos    CHECK (test_samples  > 0)
);
CREATE UNIQUE INDEX idx_mm_model_version ON model_metrics(model_version);
CREATE INDEX idx_mm_is_best    ON model_metrics(is_best);
CREATE INDEX idx_mm_model_type ON model_metrics(model_type);

-- ------------------------------------------------------

-- Table 2: usage_data
CREATE TABLE IF NOT EXISTS usage_data (
    id               BIGSERIAL       PRIMARY KEY,
    timestamp        TIMESTAMPTZ     NOT NULL,
    service_name     VARCHAR(100)    NOT NULL,
    cost_usd         NUMERIC(12, 4)  NOT NULL,
    resource_type    VARCHAR(100),
    region           VARCHAR(50),
    complexity_score NUMERIC(5, 2),
    resource_units   NUMERIC(12, 4),
    is_synthetic     BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_cost_nonneg CHECK (cost_usd >= 0),
    CONSTRAINT chk_complexity_range
        CHECK (complexity_score BETWEEN 1.0 AND 10.0
               OR complexity_score IS NULL),
    CONSTRAINT chk_service_name CHECK (service_name IN
        ('compute','storage','network','database','cache'))
);
CREATE INDEX idx_ud_timestamp    ON usage_data(timestamp);
CREATE INDEX idx_ud_service_name ON usage_data(service_name);
CREATE INDEX idx_ud_ts_service   ON usage_data(timestamp, service_name);
CREATE INDEX idx_ud_synthetic    ON usage_data(is_synthetic);

-- ------------------------------------------------------

-- Table 3: predictions
```

```sql
CREATE TABLE IF NOT EXISTS predictions (
    id                    BIGSERIAL       PRIMARY KEY,
    pr_number             INTEGER,
    repository            VARCHAR(200),
    pr_sha                VARCHAR(40),
    complexity_score      NUMERIC(5, 2)   NOT NULL,
    function_count        INTEGER,
    class_count           INTEGER,
    import_count          INTEGER,
    loop_count            INTEGER,
    branch_count          INTEGER,
    predicted_cost_usd    NUMERIC(12, 4)  NOT NULL,
    baseline_cost_usd     NUMERIC(12, 4),
    cost_delta_usd        NUMERIC(12, 4),
    compute_cost_usd      NUMERIC(12, 4),
    storage_cost_usd      NUMERIC(12, 4),
    network_cost_usd      NUMERIC(12, 4),
    model_version         VARCHAR(50)     NOT NULL,
    model_type            VARCHAR(50)     NOT NULL,
    recommendation        TEXT,
    pr_comment_posted     BOOLEAN         NOT NULL DEFAULT FALSE,
    created_at            TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_complexity
        CHECK (complexity_score BETWEEN 1.0 AND 10.0),
    CONSTRAINT chk_pred_cost_nonneg
        CHECK (predicted_cost_usd >= 0),
    CONSTRAINT chk_model_type CHECK (model_type IN
        ('linear_regression','arima','random_forest')),
    CONSTRAINT fk_model_version FOREIGN KEY (model_version)
        REFERENCES model_metrics(model_version)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
CREATE INDEX idx_pred_created_at    ON predictions(created_at);
CREATE INDEX idx_pred_pr_number     ON predictions(pr_number);
CREATE INDEX idx_pred_repository    ON predictions(repository);
CREATE INDEX idx_pred_model_version ON predictions(model_version);
```

## B Sample Data — `model_metrics`

Representative rows after a completed Sprint 2 training run:

| model_version | model_type | mae | rmse | r2 | mae_pct | is_best |
|---|---|---|---|---|---|---|
| v1_linear_regression | linear_regression | 12.431 | 18.765 | 0.782 | 9.85% | FALSE |
| v1_arima | arima | 14.210 | 21.340 | 0.734 | 11.27% | FALSE |
| v1_random_forest | random_forest | 8.921 | 13.478 | 0.891 | 7.08% | TRUE |

*ShadowStack — Dataset & Database Schema Design · v1.0 · February 24, 2026*
*BCA 6th Semester · Aniruddha Bhide · Sanay Krishna · Arpita M*
*Reviewed by: Aniruddha Bhide. Living document — subject to revision in Sprint 2.*